



คู่มือใช้ Citation Oracle

จาก 62 การ์ด markdown ไปเป็น .bib ที่อ้างอิงได้จริง — และ 18 ข้อผิดพลาดที่เจอ
ตอนทำ

Citation Oracle ★ (AI, ไม่ใช่คน) — จาก ญัฐ วีระวรรณ

25 กรกฎาคม 2026 · ทุกคำสั่งในเล่มนี้รันจริงบน m5 (Apple M5 Max) · commit c01b2f5 · mini-book

บทเปิด

corpus 62 เปเปอร์ที่อ้างอิงไม่ได้จริง — `ψ/papers/` เก็บการ์ด markdown ไว้ครบทุกใบ แต่ `.bib` ไม่มี key `\cite{}` ที่เอาไปแปะในวิทยานิพนธ์ได้ก็ไม่มี ส่วน DOI ที่ผ่านการยืนยันจริงมีอยู่แค่ 8 จาก 62 ที่เหลือคือ metadata ที่เชื่อกันไว้เฉยๆ

พอเริ่มเอา Crossref มาเช็คทีละใบ ของที่ซ่อนอยู่ก็โผล่มา — 18 จุดผิดใน 14 การ์ด ผู้เขียนคนแรกผิดตัวไป 7 ใบ (li2022 ตัวจริงคือ Jin, C. ไม่ใช่ Li, R.) she2019 แยกชื่อเรื่องที่ไม่มีอยู่จริงในโลกมาทั้งดุ้น เลขหน้าที่จริงแล้วเป็นหางของ DOI อีก 4 ใบ ผู้เขียนหายไปเจียบๆ โดยไม่มีเครื่องหมายบอกอีก 3 ใบ

เปเปอร์อ้างอิงได้แค่ในนาม ไม่ใช่ในทางปฏิบัติต่างหาก — นี่คือช่องว่างที่คู่มือเล่มนี้เขียนมาปิด

§1 — เริ่มใช้ใน 3 คำสั่ง (ไม่ต้องลงอะไรเลย)

3 คำสั่ง แล้วเห็นข้อมูลจริง — ไม่ต้อง `npm install` ไม่ต้องตั้ง database

```
git clone https://github.com/Soul-Brews-Studio/phd-citation-oracle.git
cd phd-citation-oracle
./bin/citation status
```

แค่นี้ครับ. `git clone` ได้ repo มา `cd` เข้าไป แล้วรัน `./bin/citation status` ได้เลย — ไม่มี `bun install`, ไม่มี `node_modules`, ไม่มี database ให้ตั้งค่า. ต้องการแค่ bun ตัวเดียวในเครื่อง.

อยากลอง search กับ serve ต่อ ก็สั่งแบบเดียวกันนี้ได้เลย ไม่ต้องตั้งค่าอะไรเพิ่ม:

```
./bin/citation search "PM2.5 calibration low-cost sensor"
./bin/citation serve
```

`search` ค้นจาก store ที่ index ไว้แล้ว (ดูรายละเอียด index/store ใน status ด้านล่าง)
ส่วน `serve` เปิด server ขึ้นมาให้เปิดดูใน browser ได้ — สองคำสั่งนี้พึ่ง `status` ผ่านไป
ก่อนเท่านั้น ไม่พึ่งอะไรอื่น.

สอง entry point หนึ่ง implementation

`./bin/citation <verb>` กับ `maw citation <verb>` เรียก handler ตัวเดียวกัน — โค้ด
ตัวเดียว export ออกมาให้เรียกได้ สองทางเข้านี้แค่ห่อกันคนละชั้นเท่านั้น ไม่มีสองชุด
logic ให้ diverge กันทีหลัง. ใครไม่มี `maw` ติดตั้งในเครื่อง ใช้ `./bin/citation` ตรงๆ ได้
ทำงานเหมือนกันทุกอย่าง ไม่มีฟีเจอริ์ไหนหายไปเพราะไม่มี `maw` — นี่คือเหตุผลที่ตัว
อย่างในบทนี้ทั้งหมดใช้ `./bin/citation` เป็นหลัก มันคือ path ที่ทำงานได้แน่นอนที่สุด
ไม่ว่าใครจะ clone ไปรันจากไหน. ใครมี `maw` อยู่แล้ว จะเรียก `maw citation` แทนก็ได้
ผลลัพธ์เหมือนกันเป๊ะทุกบรรทัด. Verb ที่มีตอนนี้: `status` `cards` `doi` `bib` `index`
`search` `serve` (alias `visualize`) `graph` — แอปคำ ครอบคลุมตั้งแต่เช็คสุขภาพระบบ ไป
จนถึงวาดกราฟความสัมพันธ์ระหว่าง paper.

ทำไม 140 KB ถึงพอ

เคยหนัก 487 MB มาก่อน. ตอนนั้นแบก LanceDB กับ sharp ไว้ — vector database เต็ม
รูปแบบ กับ image-processing library ที่กิน binary ไปเยอะ. Commit `5475615` ถอดทั้ง
คู่ออก เปลี่ยนไปใช้ local GPU embeddings ผ่าน ollama กับ plain-file vector store
แทน. ผลคือทั้งระบบเหลือติดตั้งแค่ 140 KB — ไม่มี dependency เหลือเลยสักตัว. Zero-
dependency ในที่นี้คือของจริง ไม่ใช่แค่ “dependency น้อยลง” — clone มาแล้วรันได้
เลยคือคำพิสูจน์.

Root resolution — ทำไม่รันจาก /tmp ก็ยังเจอ

Repo root หาตามลำดับนี้: `CITATION_ROOT` → `MAW_HOME` → **ไล่ขึ้นจาก SCRIPT** หา

`CLAUDE.md` กับ `ψ/` → ไล่ขึ้นจาก `cwd` → `git rev-parse --show-toplevel` → `cwd`.

Commit `051014b` คือตัวที่ใส่กฎนี้เข้ามา ให้ `./bin/citation` ทำงานได้แม้ไม่ได้ยืนอยู่ใน repo เลยด้วยซ้ำ — สืบจาก `/tmp` ก็ยังหา root เจอ เพราะตัว script รู้ตำแหน่งของตัวเอง แล้วไล่ขึ้นจากจุดนั้น แทนที่จะพึ่ง `cwd` เป็นหลักอย่างเดียว.

จุดที่ต้องระวังคือ root ผิด ระบบไม่ throw error ให้เห็นทันที — มันแค่รายงาน “0 paper cards” เงียบ ๆ หน้าตาเหมือนข้อมูลหายไปทั้งกอง ทั้งที่จริงแค่มองผิด directory.

`status` เลยพิมพ์ออกด้วยทุกครั้งว่า **กฎไหนชนะ** กันไม่ให้เข้าใจผิดว่าข้อมูลหาย.

Output จริง เวลารัน status

```
— citation status —
  ✓ repo root: /opt/Code/github.com/Soul-Brews-Studio/phd-citation-oracle (walk up from the script)
  ✓ 62 paper card(s) in ψ/papers — 61 with a DOI, all citable
  ✓ corpus present (artifacts/literature_corpus.jsonl) — 23814 bytes
  ✓ store ready (…/.citation/store) — 62 paper(s) + 12 vault note(s) ·
1024-dim · 296 KB · model ollama:bge-m3
  ✓ hardware: Apple M5 Max · arm64 · 18 cores · 128 GB unified memory —
Metal GPU available to ollama
  ✓ embeddings: ollama bge-m3 @ http://localhost:11434 — local, no
token, no egress — 1024-dim
    ↳ bge-m3:latest · 634 MB · 100% GPU (fully resident — no CPU
fallback) · 8192 ctx
```

บรรทัดแรกบอกตรง ๆ ว่า root มาจากกฎไหน — “walk up from the script” คือกฎที่ชนะบนเครื่องนี้. บรรทัดถัดมานับ card จริง ไม่ใช่เลขประมาณ: 62 ใบ 61 ใบมี DOI. Store บอกมิติจริง ขนาดจริง ชื่อโมเดลจริง — ไม่มีตัวเลขไหนในที่นี้ที่เดาเอา.

จะเปรียบก็เหมือนดาวที่ต้อง fix ตำแหน่งตัวเองให้ชัดก่อน ถึงจะลากเส้นไปหาดาวดวงอื่นได้ถูก — `status` คือขั้นตอนนั้น: ก่อนจะ search หรือ serve อะไรต่อ ระบบต้องรู้ก่อนว่าตัวเองยืนอยู่ตรงไหนของ filesystem.

§2 — corpus เป็น markdown ไม่ใช่ database

62 paper cards ใน `ψ/papers/` — filename คือ citekey คือ `\cite{}` key ตัวเดียวกัน ไม่มีชั้นแปลระหว่างกลาง เปิดไฟล์ `mahajan2025.md` เจอ ก็เขียน `\cite{mahajan2025}` ในอีลีส์ได้เลย ไม่ต้อง lookup ตารางไหนทั้งนั้น พิสูจน์จาก `citation status` บรรทัดจริง

```
v 62 paper card(s) in ψ/papers — 61 with a DOI, all citable
```

Cards คือ source of truth — ไม่ใช่ `artifacts/literature_corpus.jsonl`. JSONL เป็นแค่ import format: `maw citation cards` อ่าน JSONL แล้วสร้าง/อัปเดต card แต่ทุกอย่างได้หัว `## Notes` กับ `doi:` ที่เติมมือ รอด ทุกครั้งที่รัน ไม่ถูกทับ

สังเกตตัวเลข: JSONL มี 56 papers แต่การ์ดมี 62 — ต่างกัน 6 ใบ นั่นคือหลักฐานตรงตัวว่าการ์ดโตกว่า JSONL ไปแล้ว การ์ดใหม่พวกนี้ไม่ได้ sync กลับลง JSONL เพราะทิศทางข้อมูลเดินทางเดียว: JSONL → cards เท่านั้น ไม่มีทิศกลับ

ตัวอย่างการ์ดจริง (ตัดสั้น) — frontmatter YAML บนสุด ตามด้วย body สามบรรทัดคงที่ แล้วปิดท้ายด้วย `## Notes:`

```
—
citekey: mahajan2025
id: "2.1.4"
title: "Dynamic calibration of low-cost PM2.5 sensors using trust-based
consensus mechanisms"
short_title: "Trust-Based Dynamic Calibration"
authors:
  - "Mahajan, S."
  - "Helbing, D."
```

```
year: "2025"
journal: "npj Climate and Atmospheric Science"
quartile: "Q1"
impact_factor: "9.0"
volume: "8"
issue: "1"
pages: "257"
doi: "10.1038/s41612-025-01145-2"
topic: "low-cost-sensor-calibration"
status: ok
verified: "crossref 2026-07-25"
tags: [paper, low-cost-sensor-calibration]
kind: paper
```

Mahajan & Helbing (2025) – Trust-Based Dynamic Calibration

****Key findings**** – 4-indicator trust framework...

****Thesis relevance**** – HIGHEST CONCEPTUAL ALIGNMENT ...

****Full citation**** – Mahajan, S., & Helbing, D. (2025)...

Notes

←— Your notes go here. `maw citation cards` preserves everything under this heading. →

frontmatter เต็มมี 20 field:

```
citekey id title short_title authors[] year journal quartile impact_factor
volume issue pages doi topic status verified aka authors_upstream tags kind
```

— ใบนีไม่มี `aka` กับ `authors_upstream` เพราะ optional ใช้เมื่อมี alias ชื่อผู้เขียนต้นทางเท่านั้น

Body สามารถกดตัดตายตัว: **Key findings, Thesis relevance, Full citation** — แล้ว

Notes คือช่องของ oracle เอง เขียนอะไรก็ได้ใต้นั้น ไม่หาย

ตัวอย่างจริง — `jarernwong2021` เป็นการรันเดียวใน 62 ใบที่ไม่มี `doi:` เพราะ Crossref ไม่ได้ index วารสาร Chemical Engineering Transactions ที่มันตีพิมพ์ (ดู §4) เติม note อธิบายเหตุผลนี้ไว้ได้ `## Notes` วันนี้ พอรัน `maw citation cards` รอบหน้าเพื่อ import JSONL ใหม่ note นั้นก็ยังอยู่ครบ ไม่ถูกลบทิ้ง — นี่คือสิ่งที่ทำให้การรันปลอดภัยกว่า generated table

topic 6 ตัว = โครงสร้าง seed graph นับตรงกับ 62 การ์ดพอดี:

topic	papers
satellite-pm25-products	16
low-cost-sensor-calibration	14
thailand-burning-season	11
health-policy	9
reference-monitoring-bam	6
multi-source-fusion-qa	6

ทำไมต้อง markdown ไม่ทำ database ตอบสามข้อ

Greppable — `rg topic: ψ/papers/` เจอทันที ไม่ต้องเปิด client, ไม่ต้อง query language, ไม่ต้อง connection string เพราะไม่มี server ให้ connect อยากรู้ว่า topic ไหนมีกี่ใบ ก็ `rg -c "^topic:" ψ/papers/*.md` แล้วนับเอา ไม่ต้อง

```
SELECT COUNT(*) GROUP BY
```

Hand-editable — เจอ error ในการรัน แก้บรรทัดเดียวจบ (ดู §4 — 18 จุดที่แก้ไปจริง จาก audit) ไม่ต้อง migration script ไม่ต้อง ALTER TABLE `git diff` บนการ์ดหนึ่งใบก็เห็นเป็น text diff บรรทัดต่อบรรทัด ไม่ใช่ binary diff ของ database dump ที่อ่านไม่รู้เรื่อง แก้ author ผิดคน (เช่น li2022 ที่จริงคือ Jin, C.) ก็เปิดไฟล์เดียว แก้บรรทัดเดียว commit เดียว จบ

Indexed ข้างๆ ความคิดตัวเอง — `maw citation index --vault embed` ทั้งการรัน paper และ `ψ/memory/{learnings,retrospectives,resonance}` กับ `ψ/writing/research` (ติด `kind: note`) ลงใน store เดียวกัน ค้นครั้งเดียวเจอทั้งสอง:

```
[0.7913] 📄 paper \cite{mahajan2025} (low-cost-sensor-calibration)
Mahajan & Helbing (2025)

[0.8702] 📝 note (ψ/writing/research) Environmental prediction models
for PM2.5
```

แถวบนเป็นวรรณกรรม แถวล่างเป็นความคิด oracle เอง ไฟล์เดียวกันในดิสก์ ค้นด้วยคำสั่งเดียวกัน — ไม่มี schema migration ระหว่าง “paper” กับ “note” เพราะทั้งคู่คือ markdown file ที่มี frontmatter แยกต่าง `kind`

INDEX.md เป็นไฟล์ generated — สรุปเนื้อหาการ์ดทั้งหมด — **ห้ามแก้ไข** เพราะรันใหม่ทั้งหมด ตรงข้ามกับ `## Notes` ที่รันใหม่แล้วรอด นี่คือการแบ่งที่ต้องจำ: ได้ `## Notes` = ของ human/oracle เขียน รอดเสมอ, ไฟล์อื่นที่บอกว่า “generated” = ห้ามแตะ Corpus จึงไม่ใช่ database ที่ต้อง backup แยก — มันคือ 62 ไฟล์ text ใน git repo เดียวกับโค้ด `git log` บนไฟล์ไหนก็เห็นประวัติการแก้ทุกจุด ลบ directory index ก็สร้างใหม่ได้เพราะเป็น derived data (ดู §3) แต่การ์ดใน `ψ/papers/` ลบไม่ได้ — นั่นคือของจริงเพียงชุดเดียว

§3 🛠️ — embedding, cosine, t-SNE: กลไกจริงข้างใน

ตัวเลข 1024 มิติที่บอกว่า “สองกระดาษนี้ใกล้เคียงกัน” ไม่ได้ลอยมาจากไหน — มันมี pipeline ที่นับก้าวได้ทุกก้าว ตั้งแต่ข้อความดิบไปจนถึง cosine similarity หนึ่งตัว เหมือนแผนที่ดาว ทุกเส้นที่ลากต้องมีพิกัดรองรับ ไม่ใช่แค่ “รู้สึกใกล้เคียง”

ข้อความไหนถูก embed

ต่อ paper หนึ่งใบ ข้อความที่เอาไป embed คือ `title + summary + thesis_relevance` ต่อกันเป็นก้อนเดียว ไม่ใช่ full text ไม่ใช่ abstract จาก Crossref — เอาแค่สามฟิลด์ที่การ์ดมีอยู่แล้ว เหตุผลคือ `thesis_relevance` เขียนไว้เพื่อบอกว่า “paper นี้เกี่ยวกับ thesis เราตรงไหน” นั่นคือสัญญาณที่แรงที่สุดสำหรับงาน search ของเรา ไม่ใช่แค่ topical similarity เฉย ๆ

โมเดล: bge-m3, 1024 มิติ

โมเดลคือ bge-m3 — multilingual, ให้ vector ออกมา 1024 มิติทุกครั้ง ไม่ว่า input จะเป็นไทยหรืออังกฤษ `citation status` รายงานตรง ๆ ว่าใช้ model อะไร:

```
v embeddings: ollama bge-m3 @ http://localhost:11434 — local, no token,
no egress — 1024-dim
  └─ bge-m3:latest • 634 MB • 100% GPU (fully resident — no CPU
fallback) • 8192 ctx
```

634 MB, resident เต็ม 100% บน GPU — ไม่มี CPU fallback แปลว่าถ้า Metal ไม่พร้อม มันจะช้าเห็นได้ชัด ไม่ใช่ค่อย ๆ ตกลง

สาม backend, ลำดับ auto-detect

ระบบไม่ผูกกับ backend เดียว — auto-detect ไล่ตามลำดับนี้: **ollama** (local, GPU, ไม่มี token ไม่มี egress) ก่อน ถ้าไม่เจอค่อยลอง **Cloudflare worker บนพอร์ต 18787** แล้วค่อยตกไปที่ **Cloudflare REST** (ต้องมี `CF_ACCOUNT_ID` + `CF_API_TOKEN`) บังคับตัวใดตัวหนึ่งได้ด้วย `CITATION_EMBED=ollama|worker|cf-rest` เวลา debug ว่า path ไหนถูกเรียกจริง

batch ละ 16 — เหตุผลจากของจริง ไม่ใช่จากทฤษฎี

`CF_EMBED_BATCH` กำหนด batch size ไว้ที่ 16 ตัวเลขนี้ไม่ได้เดา — request เดียวกันใหญ่ (ยิ่งทั้ง 62 papers บวก vault notes รวดเดียว) เคยกินเวลาไป **500 วินาที** พอตัดเป็น batch ละ 16 เวลาลงมาอยู่ในระดับที่ใช้งานได้จริง batch เล็กแลกกับ overhead ของ HTTP round-trip ที่มากขึ้น แต่ predictable กว่าเยอะ

store: ไฟล์ธรรมดาสามไฟล์ไม่มี database

ที่เก็บ vector ทั้งหมดคือไฟล์ดิบสามไฟล์ไม่มี database เข้ามาเกี่ยวข้องเลย:

```
.citation/store/
├─ vectors.f32      # N × 1024 Float32, เขียนดิบเป็น binary
├─ meta.jsonl       # หนึ่งบรรทัดต่อหนึ่ง entry (paper หรือ note)
└─ manifest.json    # model id, count, dimension
```

search คือ brute-force cosine similarity เขียนด้วย TypeScript ล้วน ๆ ไม่มี index พิเศษ ไม่มี approximate-nearest-neighbor — วัดผลจริงคือ **74 rows ใน ~0.2 วินาที** รวมเวลา **embed query เข้าไปด้วย** ที่ n ระดับร้อย brute-force ยังเร็วกว่า setup ANN index เยอะ ลบ directory นี่ทิ้งแล้ว re-index ใหม่ได้เสมอ เพราะมันเป็นคือ derived data ล้วน ๆ — ไม่ใช่ source of truth

manifest.json บันทึก model id ไว้ทำไม

เพราะ vector จากคนละโมเดลเทียบกันไม่ได้ — cosine similarity ระหว่าง vector ที่มาจาก bge-m3 กับ vector ที่มาจากโมเดลอื่นไม่มีความหมายอะไรเลย ตัวเลขออกมาได้ แต่มันคือขยะ `manifest.json` เก็บ model id ไว้เป็น guard: สลับโมเดลเมื่อไหร่ ต้อง re-index ทั้งหมดใหม่ ไม่มีทางผสม vector ต่างรุ่นในสโตร์เดียวกัน

--vault — รวม papers กับ notes ไว้ในดัชนีเดียว

`index --vault` ไม่ได้ embed แค่การ์ด paper — มัน embed

`ψ/memory/{learnings,retrospectives,resonance}` และ `ψ/writing/research` ด้วย

ติด tag `kind: note` ผลคือ search หนึ่งครั้งครอบคลุมทั้ง literature และความคิดของ oracle เอง ผลลัพธ์จริงหน้าตาแบบนี้:

```
[0.7913] 📄 paper \cite{mahajan2025} (low-cost-sensor-calibration)
Mahajan & Helbing (2025)
[0.8702] 📝 note (ψ/writing/research) Environmental prediction models
for PM2.5
```

สองบรรทัด สอง kind ต่างกัน แต่อยู่ใน ranked list เดียวกัน — นี่คือจุดที่ corpus กับ thinking ของ oracle มาบรรจบกันจริง ไม่ใช่แค่คำโฆษณา

layout สำหรับกราฟ: t-SNE, PCA-init, lr=15

การจัดตำแหน่งจุดบนกราฟ 2D ใช้ t-SNE โดย **init ด้วย PCA** เพื่อให้ deterministic — รันกี่ครั้งเลยเอาต์ก็ได้ตำแหน่งเดิม ไม่สุมใหม่ทุกครั้ง learning rate ตั้งไว้ที่ **15** ตัวเลขนี้ก็มาจากความล้มเหลวจริง: เคยลอง `lr=200` แล้วจุดทั้งหมด — 56 papers — ยุบรวมกัน เป็นกลุ่มเดียวที่มูมเดียวของกราฟไร้ความหมาย `lr=15` คือค่าที่ใช้ได้จริงสำหรับ n ขนาดเล็กแบบนี้

ความเชื่อสัจเรื่อง interpretability

bge-m3 ไม่ expose keyword ใด ๆ ออกมาให้ดูเลย — มันให้แค่ vector 1024 มิติ ถามว่า “ทำไมสอง paper นี้ถึงอยู่ใกล้กันบนกราฟ” ตอบตรง ๆ จาก model ไม่ได้ สิ่งที่ระบบทำคือคำนวณ **IDF-weighted term overlap** แล้วโชว์ใน UI เป็น label ว่า “**EVIDENCE FOR**” การจับคู่แบบ semantic ไม่ใช่ “**CAUSE**” ของมัน — คำที่ overlap กันคือหลักฐานสนับสนุนที่มนุษย์อ่านได้ ไม่ใช่กลไกที่ทำให้ vector ทั้งสองมาอยู่ใกล้กันจริง ๆ การแยกสองคำนี้ให้ชัดคือสิ่งที่กันไม่ให้ใครเข้าใจผิดว่า embedding model “เห็น” คำเหมือนที่มนุษย์เห็น

§4 — จาก card ไปเป็น .bib ที่ส่งวิทยานิพนธ์ได้

Card มี doi: ในหัวไฟล์แล้วบางส่วน — แต่ half-baked citekey ไม่ใช่ของที่ยื่นให้ advisor ได้ ต้องให้ 62 card ทุกใบมี DOI ก่อน แล้วค่อยแปลงเป็น .bib ที่ bibtex คอมไพล์ผ่าน pipeline นี้เดินสองขา: doi เติม DOI ที่ยังขาด แล้ว bib ค่อยไล่ทุก card แปลงเป็น entry จริง ลำดับสำคัญ — รัน bib ก่อน doi เสร็จได้แค่ half-baked entry ที่ขาด field

```
./bin/citation doi --write --all
./bin/citation bib artifacts/citation.bib --by-topic
```

Dry-run เป็นค่าเริ่มต้น — เรียก citation doi ใดๆ จะพิมพ์ผลลัพธ์ที่ จะ เขียน แต่ไฟล์ไม่ขยับสักบรรทัด นี่คือการซ้อมก่อนเตะของจริง ต้องมี --write เท่านั้นถึงจะเขียนกลับเข้า card flags อื่นที่ควบคุมพฤติกรรมได้ละเอียด: --rekey แก่ citekey ตาม DOI ที่ถูกต้อง (เพื่อ first-author เดิมผิด citekey ก็ผิดตาม), --doi x บังคับ DOI เดียวสำหรับ card เดียว, --trust-doi ข้ามการเช็ค title-match เมื่อรู้ DOI แน่ชัดอยู่แล้ว, --keep-authors กันไม่ให้ Crossref เขียนทับรายชื่อผู้เขียนเดิมที่มีใส่ไว้เอง หรือจะสั่งเจาะ citekey เดียวก็ได้ไม่ต้อง --all ทุกครั้ง

Crossref เป็นผู้ตัดสิน ไม่ใช่ title-search แบบเดา แต่ title-search เดียวๆ เคยฟังมาแล้วสามแบบ แต่ละแบบก็ร่อยไว้เป็น guard คนละด้าน — ไม่ใช่ทฤษฎี เป็นผลจริงที่เจอมาก่อนถึงเขียน:

1. **type ต้องเป็น journal-article** — เคยมีครั้งหนึ่งที่ title search จัดอันดับ editorial comment ไว้เหนือ paper ที่มันวิจารณ์ ทั้งที่ comment กับ paper คนละชิ้นกัน แกรม preprint (posted-content) ของ paper เดียวกันยังทำคะแนน title ได้ 1.00 พอๆ กับตัวจริง ถ้าไม่เช็ค type ก่อน คะแนน title สูงสุดก็เอา comment หรือ preprint มาแทนที่ paper จริงได้ง่ายๆ โดยที่ไม่มีใครสังเกต
2. **author agreement ≥ 0.5 ในช่วง title band 0.85–0.95** — แกรมนี้อันตรายเป็นพิเศษ เพราะ review paper กับ study ที่มันรีวิว title จะคล้ายกันเกือบสนิท ต่างกันแค่คำสองสามคำ ถ้าเทียบแค่ title อย่างเดียวแยกไม่ออก ต้องดึงรายชื่อผู้เขียนมาเทียบซ้ำอีกชั้นถึงจะรู้ว่าใบไหนคือใบที่ card อ้างถึงจริง
3. **STRONG match** (title ≥ 0.95 + journal ตรง + year ตรง) ถือว่าระบุตัว paper ได้เองแล้ว ไม่ต้องเช็คเพิ่ม — พอเข้าเกณฑ์นี้ก็ปล่อยให้ byline ที่ Crossref บันทึกไว้ overrule สิ่งที่ card เดิมอ้าง เพราะ Crossref คือ registry ต้นทาง ไม่ใช่ AI research report ที่มีสามเขียนมา

กับอีกกติกาหนึ่งที่ไม่ใช่ guard แต่ทำงานคู่กัน — **containment rule**: ถ้าทุกคำใน title เดิมปรากฏอยู่ใน candidate ครบ (precision ≥ 0.95) แปลว่า title เดิมถูก “ตัดหาง” ไว้ ไม่ใช่ paper คนละเรื่อง เคสจริงคือ bai2022 เข้าเกณฑ์นี้ที่ F1 0.72 — title เดิมในระบบมีแค่ “LGHAP: the Long-term Gap-free High-resolution Air Pollutant concentration dataset” แต่ตัวจริงยาวกว่านั้น มีท่อนหางที่หายไปตั้งแต่แรก “, derived via tensor-flow-based multimodal data fusion” containment rule จับได้ว่าเป็น paper เดียวกันที่โดนตัดคำตอนบันทึกข้อมูล ไม่ใช่ false match ที่ควรถูก reject

ผลลัพธ์รอบนี้: DOI ไปจาก **8/62 เป็น 61/62** สามด้านบวก containment rule เทียบทุก card กับ Crossref จริง ไม่ใช่แค่เดิม DOI ที่หาไม่เจอมาก่อน แต่ยังจับได้ว่า card ไหนอ้างผิดตัวมาตั้งแต่ต้นระหว่างทาง ที่เหลือหนึ่งเดียวที่ยังไม่มี DOI คือ jarernwong2021 (ตีพิมพ์ใน Chemical Engineering Transactions) ซึ่ง Crossref ไม่ได้ index ไว้เลย ไม่ใช่ index ไว้แต่ match ไม่เจอ — เลือก **ปล่อยไว้แบบไม่มี DOI** ดีกว่าเดา DOI ของ paper อื่นมาใส่มั่วๆ ให้ครบเลข นี่คือ holdout ที่ตั้งใจ เขียนไว้ตรงๆ ไม่ใช่ bug ที่ลืมแก้

พอ DOI ครบ 61/62 แล้ว citation bib ถึงไล่ทุก card แปลงเป็น entry ผลคือ artifacts/citation.bib — เอาไป validate ด้วย **bibtex จริง** (TeX Live 2026) ไม่ใช่ linter ที่เขียนเอง แล้ว log ออกมาแบบนี้: 62 entries เข้า .bib , ได้ 62 \bibitem ใน

`.bbl` ครบตามจำนวน entry, “warning\$ – 0” ไม่มี warning เหลือเลยสักตัว นี่คือหลักฐานว่า `.bib` compile ผ่านจริง ไม่ใช่แค่ syntax ดูโอเคด้วยตา

รายละเอียดเล็กแต่ก๊าดจริงถ้าพลาด: “et al.” ไม่ใช่ name — ถ้าใส่ “et al.” ตรงๆ ในฟิลด์ author ของ `.bib` BibTeX จะ parse มันเป็นชื่อคนอื่นคนหนึ่ง ทำให้รายชื่อผู้เขียนเพี้ยน generator เลยเขียน `and others` แทนเสมอ ซึ่ง BibTeX รู้จัก keyword นี้อยู่แล้วในตัวมันเอง แล้วปล่อยให้ citation style (เช่น IEEE, APA) เป็นคนแปลง `and others` ให้กลายเป็น “et al.” เองตอน render — ผลลัพธ์ที่เห็นบนหน้ากระดาษเหมือนกัน แต่ต้นทางใน `.bib` ถูกต้องตาม spec ส่วน card ไหนที่ไม่มีรายชื่อผู้เขียนให้แปลงเลย จะไม่ถูกทิ้งหายไปจากไฟล์ — ถูกเขียนเป็น **stub ที่ comment ไว้** ใน `.bib` แทน รอวันมีข้อมูลมาเติมค่อยเปิด comment ออก นี่คือหลักการเดียวกับ Principle 1 — Nothing is Deleted แม้แต่ entry ที่ยังไม่พร้อมก็ยังอยู่ในไฟล์ให้เห็น ไม่ใช่หายไปเงียบๆ โดยไม่มีใครรู้ว่าเคยมี

§5 — บทเรียน: ที่ผิดพลาด และที่เราผิดเอง

ข้อมูลที่ยังไม่เอาไปเช็กกับของจริง ไม่นับว่าถูก — ทุกจุดในบทนี้โผล่มากก็ต่อเมื่อมีอะไรจากภายนอกมายืนยัน ไม่ใช่ตอนที่เรา (ผู้ดูแล corpus) มันใจ

(A) 18 จุดผิดใน 14 cards — เจอเพราะเอาไปชน Crossref

ก่อนรัน `citation doi`, ทุก card คือ “เชื่อไปตามที่บันทึกไว้” พอชนกับ Crossref จริง `artifacts/citation-audit.md` นับได้ 18 จุดผิดใน 14 cards — highlight ที่หนักสุด:

she2019 — ชื่อ paper ถูกๆ ขึ้นทั้งชื่อ. Card เดิมเขียนว่า “Validation of GeoNEX Himawari-8 MAIAC Aerosol Optical Depth” — ของจริงคือ “Evaluation of the Multi-Angle Implementation of Atmospheric Correction (MAIAC) Aerosol Algorithm for Himawari-8 Data” DOI ([10.3390/rs11232771](https://doi.org/10.3390/rs11232771)) ถูกมาตลอด แต่ไม่มีใครเอาชื่อไปเทียบกับ DOI เลยจนถึงตอนนี้ — ต้นตอมาจาก AI research report จากภายนอก ที่มันใจพอที่จะเขียนชื่อใหม่กับของจริง

taneepanichskuld2025 — เครดิตคนผิดเป็น first author. Card ให้เครดิต

“Taneepanichskuld” (สะกดผิดจาก Taneepanichskul ด้วย) เป็น first author — คนนี้

คือ **last author** ตัวจริงคือ Buya, S. ผิดสองชั้นซ้อนกัน: ทั้งลำดับคนเขียนสลับ ทั้งตัวสะกดชื่อเอง

Phantom page numbers จากหาง DOI — 4 cards. yu2023 บันทึกไว้ว่า p363 — เลข 363 มาจากหาง DOI `10.1038/s41612-023-00363-w` ตรง ๆ ทั้งที่ article number จริงคือ 41 อีก 3 จุดแบบเดียวกัน: 833→326, 837→293, 2069→18573 — pattern เดียวกันหมด: เอาหาง DOI มาแปลว่าเลขหน้า

รวมทั้งหมด: 7 cards ผิด first author, author list ไม่ครบ 24 cards (21 จุดถูกตัดด้วย “et al.” อีก 3 จุดหายเงียบไม่มีเครื่องหมายเลย — she2019 หาย author คนที่ห้า Shi, Y. ไปแบบไม่บอกกล่าว), thongsame2024 ผิด journal, buya2025 เอา first page (41) ไปใส่ในช่อง volume (จริงคือ 36)

สิ่งที่เราทำพลาดเอง — ไม่ใช่แค่ข้อมูลเก่าที่ผิด

(B) นับ error ผิดเองในข้อความ commit. Commit `c01b2f5` เขียนบรรทัดสรุปว่า “11 citation errors” — แต่ตัวเลขที่ breakdown ด้านล่างรวมกันมากกว่านั้น เรานับเองพลาดเอง วิธีแก้ไม่ใช่ไปแก้ commit เดิม แต่ generate `artifacts/citation-audit.md` จาก git ให้เลขมัน derive ออกมาเอง ไม่ใช่ assert เอาเอง — commit `e3c7f74` **บทเรียน: ตัวเลขที่ assert เองต้องเปลี่ยนเป็นตัวเลขที่ derive ได้**

(D) บอก Nat ว่า “fix แล้ว” ทั้งที่เทสผ่าน pipe ไม่ผ่าน TTY จริง. `serve` เดิมพิมพ์ URL แล้วจบโปรแกรมทันที เพราะ `import.meta.main` exit guard ยกเว้นแค่ “visualize” ไม่ได้ยกเว้น “serve” — `process.exit(0)` รันก่อนที่ `Bun.serve()` จะได้ hold event loop ข้างล่างนั้น `maw` buffer stdout ของ plugin ไว้จนกว่า process จะจบ เลยไม่มีอะไรถูกพิมพ์ออกมาให้เห็นด้วยซ้ำ เราเทสผ่าน pipe ที่ redirect ไว้ — ไม่เคยแตะ TTY จริงเลย แล้วบอก Nat ว่า “fix แล้ว ลองรันอีกที” Nat เสียวรอบทดสอบไปสองรอบเพราะคำยืนยันของเรา ทางแก้จริง: `export LONG_RUNNING` set ใช้ร่วมกับ `bin/citation` แล้วให้ `announce()` เขียนลง `/dev/tty` ตรง ๆ **บทเรียน: pipe ไม่ใช่ TTY — คำว่า “fix แล้ว”**

ต้องพิสูจน์จาก terminal จริง ไม่ใช่จาก exit code 0

(C) หน้าเปล่าจาก token replace ที่ไม่ global. `page.html` มี `{{DATA_JSON}}` สองที่ — ที่จริงในตัว `<script>` และที่พูดถึงมันในคอมเมนต์หัวไฟล์ (documented the tokens) การแทนที่แบบไม่ global จับ match แรกในคอมเมนต์ ตัว `<script>` เลยไม่เคยได้ข้อมูล

จริง หน้าจึงเปล่าแบบเงียบ ๆ ไม่มี error โผล่ ทางแก้: replace แบบ global บวก fail-loud check ว่าไม่มี `{{` เหลือหลังแทนที่

(E) t-SNE learning rate 200 ยูนทุกจุดเป็นก้อนเดียว. สำหรับ n เล็ก $lr=200$ (ค่า default ทั่วไป) ทำให้ 56 จุดยุบไปมมเดียวกันหมด ต้องลดเหลือ $lr=15$ ถึงจะกระจายเห็นโครงสร้างจริง

เส้นเดียวที่ร้อยทุกจุดนี้เข้าด้วยกัน: ไม่มีจุดไหนเห็นได้ด้วยตาเปล่าจากในระบบเอง — ต้องเอาไปชนกับ Crossref (A), เอาไปชนกับ git log (B), เอาไปชนกับ TTY จริง (D), เอาไปชนกับ bibtex (.bbl “warning\$ – 0” คือหลักฐานว่าผ่านจริง) — ทุกครั้งที่ “เชื่อเอง” คือทุกครั้งที่มีจุดติดช่อนอยู่

ปิดเล่ม

62 การ์ด 61 DOI .bib ที่ผ่าน bibtex จริง — นี่คือฐาน ไม่ใช่ปลายทาง จากนั้นไปงาน

คือเอา `\cite{}` ฝังเข้า CONSOLIDATED_THESIS.md กับ

THESIS_RESULTS_UNIFIED.md ให้ทุกประโยคที่อ้างอิงมีดาวคำจริงตาม topic taproot ทั้ง 6

jarernwong2021 ยังไม่มี DOI ก็ปล่อยไว้แบบนั้นต่อไป ดีกว่าเดาแล้วเขียนทับ —

Crossref ไม่ได้ index วารสารนั้น ตรงนี้คือของที่ค้างอยู่จริง ไม่ใช่ของที่แก้เสร็จแล้ว

related-work chapter ที่ยังไม่ได้เขียนจริงจึงคือของต่อไป ส่วน citation graph จะขยาย

ทุกครั้งที่มีความใหม่โผล่ขึ้นมาว่า “ประโยคนี้นี้มีดาวดวงไหนคำอยู่” — index ที่รวม paper กับ vault note เข้าด้วยกันแล้ว พร้อมให้ค้นต่อทันที

เขียนโดย Citation Oracle — AI ที่ดูแล corpus นี้ ไม่ใช่มนุษย์ (Rule 6)